Information and Coding — Trabalho Prático n.º 1

Relatório 2: Utilização de ferramentas de Inteligência Artificial

Duarte Lourenço (n.º mec. 114421) Francisco Matos (n.º mec. 113726)

Universidade de Aveiro — DETI
Outubro de 2026

1 Introdução

Este relatório descreve o uso de ferramentas de Inteligência Artificial (IA) no primeiro trabalho prático e avalia criticamente a sua eficácia. A IA foi usada em quatro frentes, o planeamento e a estruturação do trabalho, a produção de blocos de código, os testes e a depuração, e a revisão dos relatórios. Para cada uma apresentamos casos concretos, com o problema que tínhamos, o que a IA propôs, como o verificámos, se estava correto, o que mudámos e o que aprendemos.

Na maior parte dos casos a ajuda foi eficaz, mas não foi infalível, e houve propostas erradas ou incompletas que tivemos de corrigir. A ideia que os casos seguintes procuram demonstrar é que a IA acelerou o trabalho nas quatro frentes, mas as decisões de projeto e a verificação dos resultados foram sempre nossas, e foi essa verificação que apanhou os erros.

A Secção 2 identifica as ferramentas usadas e a forma como as usámos, as Secções 3 a 6 tratam as quatro frentes, a Secção 7 faz a avaliação crítica e a Secção 8 conclui.

2 Ferramentas utilizadas

Usámos a IA de quatro formas. No início, descrevemos numa *prompt* o que o enunciado pedia e analisámos as recomendações da IA sobre a organização do trabalho e as tecnologias a usar. Durante o desenvolvimento, pedimos-lhe blocos de código curtos, cada um com uma função bem delimitada, que depois integrámos e testámos, e recorremos a ela para esclarecer dúvidas pontuais sobre a matéria e sobre as ferramentas. Nos testes, pedimos-lhe que gerasse ficheiros de teste, como os ficheiros WAV de casos-limite usados para testar os codecs. Por fim, nos relatórios, a IA converteu os nossos rascunhos em ficheiros LaTeX, com a formatação, as tabelas e as referências cruzadas, e reviu o texto para corrigir a gramática e eliminar erros, primeiro secção a secção e depois numa leitura de conjunto, para confirmar que a estrutura fazia sentido. A Tabela 1 indica que ferramenta cada membro usou e para quê.

Ferramenta	Membro(s)	Utilização
Claude (Cowork)	Duarte Lourenço	Estruturação, base comum, revisão dos relatórios, alguns blocos de código e esclarecimento de matéria
Claude Code	Francisco Matos	Teste e depuração, revisão do relatório, alguns blocos de código e esclarecimento de matéria
ChatGPT	Duarte Lourenço	Esclarecimento de dúvidas pontuais

Tabela 1: Ferramentas de IA usadas, por frente de trabalho.

3 Estruturação do trabalho

No início do trabalho, a IA (Claude, no modo Cowork) foi usada como tutor. Leu o enunciado connosco, explicou os conceitos de base e propôs as linguagens, a organização do repositório e uma divisão de tarefas.

A ideia inicial era fazer tudo em Python. A IA propôs C++ para os programas e codecs, uma vez que o enunciado avalia o tempo e a memória, e Python apenas para gráficos e medições. Aceitámos a divisão, e as medições confirmaram-na em parte. No histograma, o `numpy` (0,096 s) ficou quase igual ao C++ (0,093 s), porque a operação vetoriza, e a vantagem do C++ só aparece nos codecs, onde as decisões de cada bloco são sequenciais. Aprendemos que a escolha da linguagem se justifica medindo a tarefa concreta, e não em abstrato.

Para a organização, a IA propôs fazer primeiro uma base comum e só depois dividir o trabalho, com um *branch* por tarefa e um *pull request* revisto pelo outro membro, fluxo que usámos ao longo de todo o trabalho.

4 Escrita do código

Na escrita do código, a IA foi uma ferramenta de geração de código ao serviço de decisões que eram nossas. Cada membro dividia a sua tarefa em blocos com uma função delimitada (por exemplo, a leitura do WAV, o preditor, o código de Rice), definia o que cada um devia fazer e pedia à IA uma implementação, que lhe explicava até a conseguir reproduzir. Por isso a IA escreveu a maior parte das linhas do repositório, como registam as assinaturas dos *commits*, mas nenhum bloco entrou sem passar por três filtros. Quem o pediu tinha de o perceber ao ponto de o saber explicar, o bloco tinha de passar nos testes (reconstrução comparada byte a byte com o original, métricas recalculadas em `numpy`) e tinha de ser revisto pelo outro membro num *pull request*. Quando a IA propôs alternativas, como na arquitetura do codec com perdas, tratámo-las como hipóteses a medir e não como decisões.

No codec com perdas, a IA (Claude Code, usada pelo Francisco) propôs e escreveu a cadeia completa, com MID/SIDE ortonormal, MDCT calculada por FFT, passo de quantização único com zona morta e um codificador aritmético em vez do código de Rice da Parte II. Cada peça foi verificada de forma independente. O `test_mdct` compara a MDCT rápida com a definição e confirma a conservação da energia, o SNR anunciado pelo codificador coincide com o medido pelo `wav_cmp`, e as varreduras nos 20 ficheiros de treino justificam cada escolha (Relatório 1, Tabela 6). A proposta não estava isenta de erros. O intervalo de procura do passo ia só até 2^{20}, e num ficheiro curto de 24 bits a versão inicial entregava 71 kbit/s quando se pediam 32, sem avisar. Foi a bateria de casos-limite que o detetou, e o intervalo passou a depender dos bits por amostra. Depois de o codec sem perdas passar a processar em fluxo, pedimos o mesmo para o codec com perdas, e a memória desceu de 34 para 5,4 MiB sem alterar o SNR, com os ficheiros comprimidos a ficarem iguais byte a byte aos da versão anterior para o mesmo passo.

Memória que crescia com a duração do áudio *Claude (Cowork) — Duarte Lourenço*

Problema. Quando medimos a memória pela primeira vez, com o codec sem perdas já a comprimir bem, o pico crescia com a duração do áudio, com 16,5 MiB num ficheiro de 5 MiB (contra 3,4 MiB do `flac -8`) e 157 MiB num de 44,4 MiB. A causa eram duas cópias integrais feitas no arranque, as amostras em inteiros de 32 bits, com o dobro do tamanho, e o ficheiro inteiro em bytes, só para preservar o contentor. O pico previsto, cerca de três vezes o tamanho do ficheiro, coincidia com o medido.

Proposta da IA. Como todas as decisões são locais ao bloco e o preditor só precisa das três amostras anteriores, processar em fluxo, mantendo em memória apenas um lote de blocos e três *frames* de histórico, escrevendo a saída à medida que avança, e lendo o contentor percorrendo os blocos RIFF e saltando o bloco `data`, em vez de o copiar.

Verificação. Os ficheiros comprimidos pelas duas versões, comparados com `cmp`, saem idênticos bit a bit, e os 25 casos-limite continuam a passar.

Estava correto? Sim. A memória desceu para 4,8 MiB a codificar e 4,0 MiB a descodificar, e ficou em 4,6 MiB no ficheiro de 44,4 MiB, ou seja, deixou de depender da duração. O tempo de codificação

caiu de 0,209 s para 0,098 s por ficheiro, sem alterar a taxa.
O que mudámos. Aproveitámos a mesma propriedade para decidir em paralelo os blocos de cada lote, escritos sempre pela ordem original, e o ficheiro produzido é idêntico com 1, 2, 4 ou 8 fios. Pedimos depois o mesmo tratamento para o codec com perdas.
O que aprendemos. O defeito não aparecia em nenhum teste funcional, porque o codec comprimia bem e reconstruía o original. Só apareceu quando começámos a medir uma grandeza que ainda não medíamos, e por isso todos os critérios de avaliação devem ser medidos desde cedo.

5 Testes, depuração e medição

Todos os programas tinham de ser testados, e os testes levantavam um problema próprio, porque os números que usávamos para decidir se um programa estava certo eram produzidos pelo próprio código em teste. Nesta frente, a IA foi usada para desenhar e escrever as ferramentas de verificação, e as conclusões sobre o que estava certo foram tiradas por nós a partir delas.

Confirmar que os resultados dos testes eram fiáveis *Claude (Cowork) — Duarte Lourenço*

Problema. O `wav_cmp` é o instrumento de medida de todo o trabalho, porque dá o erro da quantização e o SNR dos dois codecs. Se tivesse um erro, todos os resultados estariam errados e nenhum teste o mostraria, porque os testes usavam o próprio `wav_cmp`. O mesmo valia para as entropias do `wav_hist`.
Proposta da IA. Uma segunda implementação, independente da primeira, em Python com `numpy`. O script lê os WAV com o módulo `wave` do Python, sem a `libsndfile` nem o nosso leitor, recalcula as entropias e os desvios padrão de L, R, MID e $(L-R)/2$, o MSE, o L^∞ e o SNR de ficheiros quantizados, e confirma a reconstrução MID/SIDE em todas as amostras. Cada valor é comparado com o do C++ com uma tolerância fixa, e o script termina com erro se alguma comparação falhar.
Verificação. Corremos o script nos 20 ficheiros de treino. Como usa outro leitor de WAV e outro algoritmo (`numpy.unique` em vez da contagem do C++), um erro só passaria se as duas implementações errassem da mesma maneira.
Estava correto? Sim. Os valores coincidiram dentro das tolerâncias em todos os ficheiros, e o recálculo passou a ser um dos quatro caminhos de verificação descritos no Relatório 1.
O que mudámos. Nenhuma no código em C++, que estava certo.
O que aprendemos. Duas implementações só são independentes se diferirem naquilo que pode errar, e por isso o leitor e o algoritmo tinham de ser outros. Aprendemos também o limite desta verificação, porque as duas implementações concordam mesmo quando a pergunta está errada. Ambas calculavam a entropia de $(L - R)/2$, e nenhuma podia mostrar que a poupança do MID/SIDE devia ter sido medida com $L - R$. Um teste confirma o cálculo, não a escolha do que se calcula.

Medições de tempo que se contradiziam *Claude (Cowork) — Duarte Lourenço*

Problema. O tempo de execução é critério de avaliação, e precisávamos de saber se cada versão nova do codec sem perdas era mais rápida do que a anterior. Duas rondas de medições feitas com o mesmo código chegaram a conclusões opostas sobre qual das versões era mais rápida.
Proposta da IA. Os *scripts* de medição foram escritos pela IA e mediam corretamente o tempo e a memória de cada programa, mas não controlavam as condições da máquina. Quando lhe mostrámos a contradição, a IA propôs verificar o modo de gestão de energia do processador.
Verificação. Lemos a frequência do processador durante as medições. Com o *governor* em `powersave`, corria entre 1,1 GHz e 1,36 GHz, variando de ronda para ronda. Fixámo-lo em `performance`, repetimos todas as medições e passámos a usar o melhor de três execuções.
Estava correto? Sim. Com o processador fixo, as rondas passaram a concordar entre si, e são essas as medições do Relatório 1.
O que mudámos. O *script* de medição verifica agora o *governor* antes de começar e, se não estiver em `performance`, avisa que os tempos não serão comparáveis e só continua com confirmação explícita. A máquina, o compilador e as versões usadas ficam registados com os resultados.

O que aprendemos. Um programa de medição correto não garante medições corretas, porque as condições em que corre fazem parte da medição. A partir daí, fixámos as condições da máquina antes de comparar qualquer tempo.

6 Escrita do relatório

Nos relatórios, a IA foi usada sobretudo na forma. Ajudou a estruturar o documento, propondo a divisão em secções, a ordem por que os assuntos aparecem e o espaço a dar a cada um, reviu o texto apontando erros gramaticais e de sintaxe, e identificou redundâncias, porque várias passagens repetiam a mesma informação em secções diferentes e acabaram por ser cortadas, o que também ajudou a respeitar o limite de páginas. Gerou ainda o código LaTeX das tabelas e dos diagramas, cuja paginação verificámos compilando, e passou os resultados dos testes e das medições para tabelas a partir dos ficheiros CSV, o que evitou erros de cópia manual dos números. A interpretação dos resultados, as conclusões e a decisão sobre o que entrava ou saía foram nossas.

Fomos nós a detetar que os resultados da Parte III que estavam no repositório tinham sido medidos nos sete ficheiros preliminares, e não nos 20 ficheiros de treino oficiais, e que parte das medições de tempo tinha sido feita numa máquina diferente da descrita no Relatório 1. As medições foram repetidas nos 20 ficheiros oficiais, e as de tempo e memória da Parte III continuaram a ser as da outra máquina, o que fica identificado na Secção 2.3 do Relatório 1. Aprendemos que a revisão por IA é útil para a forma e para a coerência do texto, mas que os números só são fiáveis quando se sabe de onde vêm.

7 Avaliação crítica

Onde foi mais eficaz. A IA foi mais eficaz na escrita do código e na estruturação dos relatórios. No código, transformou especificações nossas em implementações completas num tempo que não teríamos conseguido sozinhos, como a cadeia do codec com perdas, da MDCT ao codificador aritmético, e a passagem do codec sem perdas para processamento em fluxo, que levou a memória de 45,6 MiB para 4,8 MiB sem alterar um único bit dos ficheiros comprimidos (Secção 4). Nos relatórios, converteu os nossos rascunhos em LaTeX, gerou as tabelas diretamente a partir dos ficheiros de resultados e apontou redundâncias entre secções (Secção 6).

Onde falhou. Os maiores problemas surgiram nos testes. Os erros da IA não estavam no funcionamento normal dos programas, mas nas condições à margem, como o intervalo de procura do passo, que só falhava num ficheiro curto de 24 bits, os *scripts* de medição, que mediam bem mas não controlavam o processador, e resultados medidos nos ficheiros preliminares em vez dos oficiais. Em todos estes casos a IA produziu algo plausível e que funcionava no caso comum, e o erro só se tornou visível quando o confrontámos com um caso concreto. Foram problemas resolúveis, mas exigiram insistência, com várias iterações em que lhe apresentávamos o caso que falhava, até a correção resistir a todos os testes.

O que tornou a ajuda fiável. O que tornou a ajuda fiável foi a verificação, não a ferramenta. Usámos critérios que não dependem do código que a IA escreveu, a comparação byte a byte com o `cmp`, que não admite aproximações, os testes por propriedades, como a reversibilidade do MID/SIDE e a conservação da energia na MDCT, que valem para qualquer entrada, o recálculo independente em `numpy`, e a comparação com ferramentas externas, o `flac` e o `ffmpeg`. A isto juntou-se a revisão de cada *pull request* pelo outro membro e a regra de não aceitar código que não soubéssemos explicar. A verificação tem, no entanto, um limite que aprendemos na prática, porque confirma que o cálculo está certo, mas não que se está a calcular a coisa certa (Secção 5).

Balanço de tempo. A IA poupou-nos muito tempo. Estimamos que, sem ela, o trabalho teria exigido pelo menos o triplo do tempo. A estimativa é grosseira, mas a diferença é clara, e o tempo gasto a verificar e a corrigir as propostas foi uma fração pequena do tempo poupado. Esse tempo de verificação não foi, no entanto, dispensável, porque foi ele que garantiu que o que entregamos está correto.

8 Conclusões

A IA foi uma ferramenta eficaz nas quatro frentes, e a condição para o ser foi a mesma em todas, porque nenhuma proposta foi aceite sem ser percebida e verificada por nós. Repetiríamos a forma de trabalhar, pedindo o código bloco a bloco, testando cada bloco de forma independente da ferramenta que o escreveu e confirmando os números a partir dos ficheiros de resultados.

Num próximo trabalho faríamos duas coisas de outra maneira. A primeira é anotar o uso da IA à medida que o trabalho avança, com o problema, a proposta, a verificação e a decisão de cada caso. Neste trabalho parte desse registo foi reconstruída no fim, a partir do histórico do Git e da memória de cada um, o que custou tempo e deixou pormenores por confirmar. A segunda é medir todos os critérios de avaliação desde o início, incluindo a memória e o tempo com as condições da máquina fixadas, porque o crescimento da memória só apareceu quando a começámos a medir, e as medições de tempo só se tornaram fiáveis depois de fixarmos o processador.

Fica também a lição mais geral, que a IA acelera a escrita mas não dispensa saber o que se está a fazer. Os erros que encontrámos foram apanhados porque percebíamos o código e a matéria da cadeira o suficiente para desconfiar de um resultado.